

Discusión en grupo - SOLID Race Lab

Discusión en grupo — SOLID Race Lab

Ingeniería de Software II · UNAL 2026-2 Carlos Cuervo (@carlonox) · Andrés Franco (@AFrancoSUNAL)

Respuestas a las preguntas de discusión de cada ejercicio del taller. Código en [exercises/](#).

Ex1 — SRP: ¿por qué es mejor que Car no sepa de logging ni prints?

Porque cada responsabilidad es un motivo distinto para cambiar. Si el formato del log cambia (texto plano → JSON), o si la visualización cambia (consola → ventana gráfica), Car no debería enterarse: su único trabajo es moverse y llevar su posición.

¿Qué se rompería primero con log JSON? Antes del refactor, el `open()` y el `f.write()` vivían dentro de `Car.move()`: habría que modificar la clase que mueve los carros para ajustar algo no relacionado con el movimiento. Después del refactor, solo cambia `RaceLogger.save()`. También notamos que el log original usaba modo "a" (append) en cada tick, por lo que cada corrida contaminaba la anterior; ahora `save()` vuelca todo de una vez con "w".

Ex2 — OCP: ¿qué dice del diseño querer editar Vehicle o Track?

Que no se está usando polimorfismo: si agregar una moto exige modificar la clase base, la base no está “cerrada”. Cada vehículo nuevo implicaría editar código ya probado, con riesgo de afectar lo que funcionaba. En nuestra solución, `Motorcycle` y `Bicycle` son solo subclases nuevas: `Vehicle`, `Car`, `Truck` y `Track` quedaron intactos, y `Track` corre la carrera sin conocer ningún tipo concreto.

Ex3 — LSP: ¿qué habría que agregar a Track sin el fix? ¿Por qué es mejor arreglar la subclase?

Sin el fix, `Track.run` necesitaría programación defensiva: `try/except` alrededor de cada `r.move()` más un clamp de posición para los retrocesos. Esto contamina al llamante por culpa de un implementador que no cumple el contrato, y no escala: cada vehículo nuevo sería un posible caso especial. Arreglar la subclase (fallar es quedarse quieto) restaura el contrato una sola vez, y todo código que use vehículos puede seguir confiando en él sin verificaciones adicionales.

Ex4 — ISP: ¿qué interfaces necesitarían Submarine o Airplane? ¿Hay que tocar clases existentes?

- Submarine: Movable más una nueva Diveable (`dive()`).
- Airplane: Movable más Flyable y Refuelable (ya existen).

Ninguna clase existente se modifica. Con la interfaz gorda original, agregar un submarino habría obligado a darle `fly()` y `pedal_harder()` sin sentido; con interfaces pequeñas, cada vehículo implementa solo lo que realmente hace.

Ex5 — DIP: ¿cómo facilita la inyección testear Race sin animación?

Porque `Race` ya no construye sus dependencias, las recibe. En los tests se le inyecta un `Track` sin animación ni pausas y corredores falsos que no son ninguna clase real del módulo. Antes era imposible: `Race` creaba su propio `Track` animado internamente, así que probarlo significaba correr la animación real en la terminal.